Rapport backend — AZÒ

Ce qui a été fait aujourd'hui sur l'API (azo-backend) — pour Dorin

17 octobre 2026 · 7 commits de code poussés sur main (`ecb4be03`, `8d82013c`, `8ab4ce66`, `88e49834`, `0afb4017`, `f717e40b`, `2a2a8d82`) · 31 suites / 358 tests verts

1. En résumé

Quatre changements : **type de compte enregistré sans identité** pour les profils professionnels, **le tricycle ne livre que les gros colis**, **base nationale des lieux du Bénin** et **plaque du Speeder partiellement masquée**. `tsc` sans erreur. Aucune migration.

> **Déploiement :** le push de `main` déclenche le déploiement sur Render. Après déploiement, `/health` doit afficher `ecb4be03`. Rien n'a été testé en production.

2. Inscription des profils professionnels (`auth.service.ts`)

- `verifyOtp(phone, code, profile)` : pour `DRIVER`, `COURIER` ou `AGENCY`, si le compte n'a ni `signupKind` ni `profileCompletedAt`, le serveur enregistre `signupKind` (sans nom, sexe ni NPI). Un client déjà inscrit ne peut donc pas devenir agence : la règle `signupKind === "AGENCY"` de `providers.module.ts` reste intacte.
- Le brouillon de dossier prestataire est ouvert comme avant (`openDraftOnSignup`).
- La réponse contient maintenant `signupKind` (au niveau de la racine, à côté de `user` et `provider`) : le mobile s'en sert pour ne pas redemander l'identité à un professionnel.
- Le formulaire `POST /users/me/registration` est inchangé (client). `RegistrationCompleteGuard` (CLIENT) est inchangé : un professionnel sans identité n'a pas accès aux commandes de client.
- **Non couvert par un test dédié** (les 338 tests existants passent).

3. Le tricycle ne livre que les gros colis

- `tarification.json` : `delivery.vehicles.ZEM_TRICYCLE.onlyPackages: ["large"]` ; `PricingService.deliveryRule` renvoie `null` pour tout autre type de colis.
- Devis (`GET /deliveries/quote`) et création refusés (400, « Le Speeder tricycle est réservé aux gros colis… ») pour document, petit et moyen colis ; un gros colis reste réservé au tricycle. Course simple et coursier personnel en tricycle non touchés.
- Test : `personal-courier.spec.ts` (devis refusé doc / petit / moyen, création refusée, gros colis à 1 km = 305 F).

4. Base nationale des lieux (`places/`)

- `benin-gazetteer.json` : **17 022 lieux** couvrant les **77 communes** : 77 communes, 527 arrondissements, 1 536 quartiers, 6 373 villages, 8 289 hameaux, villes et lieux-dits (relevé élargi du 17 octobre : zones et limites

administratives en plus des points « place »). La liste soignée de Cotonou et d'Abomey-Calavi (`benin-gazetteer-core.json` , 236 lieux) est prioritaire à la fusion.

- **Sources** : OpenStreetMap (Overpass, un relevé par commune), GeoNames (villes, 77 communes, 539 arrondissements). Doublons (même nom à moins de 400 m) fusionnés ; chaque lieu est rattaché à sa commune et à l'arrondissement le plus proche (40 km maximum).
- `places.module.ts` : la recherche classe commune, ville, arrondissement, quartier, village ; sous-titre « type · arrondissement, commune » ; le géocodage inverse essaie d'abord le lieu connu le plus proche (700 m), sinon la recherche en ligne.
- **Limites** : la liste officielle des villages par arrondissement (INStaD) n'a pas pu être obtenue (site en maintenance) ; la couverture dépend de ce qu'OpenStreetMap contient (inégale selon les communes, quelques dizaines de lieux pour certaines). Les arrondissements sont placés par leur chef-lieu. Les noms ne sont pas tous orthographiés selon l'usage officiel.
- « Kpota » (quartier du Séminaire, Abomey-Calavi) ajouté à la main à la liste soignée ; absent d'OpenStreetMap en tant que quartier. Quatre communes restent peu fournies : Ouinhi (17), Tori-Bossito (24), Pobè (25), Zangnanado (31). Détail par commune : `COUVERTURE_LIEUX_BENIN_2026-10-17.pdf` .
- Tests : 6 tests `places` verts.

5. Plaque du Speeder (`rides.service.ts`)

`maskPlate` : début et fin conservés, milieu remplacé par quatre étoiles (« AB1234CD » → « AB123****CD »). Le champ reste `plateMasked` ; la plaque complète n'est pas exposée au client.

5bis. Coursier occupé (`delivery.module.ts` , commit `8ab4ce66`)

- `pending()` ne renvoie plus aucune mission à un coursier qui a une mission acceptée ou en cours, ou, pour un coursier personnel, qui est réservé par un client pour la durée choisie (`isCourierBusy`). Une réservation programmée plus tard ne bloque pas. Le refus à l'acceptation existait déjà.
- Test ajouté dans `personal-courier.spec.ts` . Non vérifié : un Speeder coursier peut-il prendre une course de Speeder en parallèle (parcours séparé).

5ter. Audit de bout en bout (base jetable, API locale)

- L'API a été lancée en local sur une base PostgreSQL **créée pour l'occasion** (toutes les migrations appliquées de zéro, puis base supprimée) et environ **120 vérifications** ont été rejouées : inscription des cinq profils, dépôt et validation des dossiers, course complète (radar, proposition, codes, position par WebSocket, paiement avec 15 %, pourboire, notes, reçu), colis et coursier personnel, agence (activation, flotte, recharge et reprise), portefeuille, assistance, notifications, lieux, location, boutique, console admin, droits d'accès.
- **Faille corrigée** (`delivery.module.ts`) : n'importe quel compte (client, Speeder de courses) pouvait accepter une mission de colis. Seuls les coursiers et livreurs **validés** le peuvent (`isApprovedCourier` ; `pending` renvoie une liste vide aux autres) ; test ajouté.
- **Nom du prestataire** (`providers.module.ts`) : un prestataire inscrit par son seul numéro n'avait pas de fiche d'identité, donc pas de prénom ni de nom dans l'application. Le nom du dossier devient son nom (`splitFullName`) dès l'enregistrement du dossier et à l'approbation ; une identité vérifiée (NPI) n'est jamais écrasée. Test ajouté.
- **Non vérifié** : paiement FedaPay réel, envoi de SMS, notifications sur téléphone fermé, écrans sur téléphone.
- **À décider** : le serveur écrit encore « Spider » dans environ 95 textes (notifications, libellés de dossier) alors que l'application dit « Speeder ».

5quater. Commissions détaillées (`admin/finance.service.ts`)

- `GET /admin/finance/commissions` renvoie pour chaque course ou livraison terminée : nom et téléphone du prestataire (nom du dossier en priorité), agence éventuelle (`independent`), client, départ et arrivée, distance, date de fin, montant, commission et pourcentage. Vérifié sur PostgreSQL : course indépendante 1 000 F → 150 F (15 %) ; employé d'agence → 30 F (3 %) avec le nom de l'agence ; livraison de 600 F → 90 F.
- Notifications de fin de course et de livraison réécrites en quatre langues : « {pct} % de la course (− {commission} F) est versé à l'entreprise AZ̀ · tu reçois +… ».

5quinquies. Distance parcourue : chaque déplacement réel (`rides/odometer.ts`)

- Sous le seuil de bruit de la position, un déplacement est désormais compté à partir de la **vitesse mesurée par le téléphone** (effet Doppler, précise au dixième de m/s) : vitesse × durée (au plus 5 s par intervalle, bornée par la position), au millimètre. À l'arrêt (vitesse sous 0,5 m/s), rien n'est compté. Un champ `cred` évite de compter deux fois ce qui l'a déjà été par la vitesse quand la position confirme le déplacement.
- `ride:progress` est émis toutes les 500 ms (avant 5 s). La base garde des mètres entiers (colonnes `Int`) : la partie inférieure à un mètre reste en mémoire et part à l'envoi suivant ; écart de prix inférieur à 0,1 F.
- Tests (`odometer.spec.ts` , 16 au total) : déplacement lent à 1 m/s compté, 0,6 m/s pendant 1 s = 0,6 m, 120 points immobiles = 0, pas de double comptage. **Non vérifié sur un trajet réel** ; les missions de coursier personnel utilisent encore l'ancien comptage (sans la vitesse).
- Le prélèvement de 15 % était déjà correct : indépendant 15 %, employé d'agence le taux de son niveau.

5sexies. Coursier personnel et livreur express

- Rayon de recherche des missions : **20 km** (`PICKUP_RADIUS_KM` , avant 10 km) ; test : 15 km visible, 25 km masqué, lieu inconnu visible.
- Cause probable du coursier personnel qui ne voyait aucune commande : sa spécialité enregistrée était « livreur express » (bug du formulaire mobile, voir rapport front). **Script de correction** : `node --env-file=.env scripts/set-courier-category.js 01XXXXXXXX PERSONNEL` (`--allow-remote` pour Render).
- Règle inchangée : un coursier ne voit que les réservations de son propre véhicule.

5septies. Livraisons : le coursier se propose, le client valide (`delivery.module.ts`)

- **Migration** `20261017150000_delivery_offers` (ajout de colonnes seulement, aucune donnée touchée) : `Delivery.offeredAt` , `offerExpiresAt` , `offerSeenAt` , `declinedCourierIds` (`TEXT[]`), `clientLat` , `clientLng` , `clientLocatedAt` . Appliquée sur une base vide, schéma et migrations alignés pour la table `Delivery` .
- **Flux** : `POST /deliveries/:id/accept` (le corps peut contenir `lat` et `lng` du coursier) fait passer la mission de PENDING à **OFFERED** pendant `offerExpirySeconds` (45 s, comme les courses) si `requireClientApproval` est actif ; sinon l'ancien comportement (MATCHED direct). Puis : `GET /deliveries/offers/pending` (rattrapage de la popup), `GET /deliveries/:id/courier-profile` , `POST /deliveries/:id/offer-seen` , `POST :id/approve-courier` (OFFERED → MATCHED ; la fenêtre de réservation du coursier personnel est fixée à ce moment, avec contrôle de chevauchement), `POST :id/decline-courier` et `POST :id/withdraw-offer` (retour en PENDING, coursier ajouté à `declinedCourierIds` et masqué de `pending`).
- **Expiration** : `DeliveryOfferExpiryTask` balaie toutes les 5 s (marge de confirmation identique aux courses). Un coursier avec une proposition en attente est « occupé » (`isCourierBusy` , vérification à l'acceptation). Le

client peut annuler pendant une proposition ; un coursier qui annule pendant une proposition la retire.

- **Temps réel** (`RidesGateway.emitToUser` , exportée par `RidesModule`) : `delivery:offered` (popup du client), `delivery:offer-ended` , `delivery:offer-approved` , `delivery:offer-seen` ; notifications poussées `NOTIFICATION_DELIVERY_OFFERED(_PERSONAL/_BODY)` , `_APPROVED(_BODY)` , `_DECLINED` en quatre langues.
- **Confidentialité** : avant validation, la liste du client ne contient ni le nom ni le téléphone du coursier ; le profil public (`courier-profile.ts`) n'expose ni téléphone ni pièces ; la plaque est partiellement masquée (`rides/plate.ts` , extraite de `rides.service.ts`).
- **Suivi du client** : `POST /deliveries/:id/client-location` (client de la mission, statuts MATCHED / IN_PROGRESS) ; `GET :id/track` renvoie la position du client **seulement au coursier** de la mission.
- **Vérifié** sur une base jetable avec l'API et des WebSocket : 22 vérifications sur 23 passent (popup temps réel, rattrapage, profil sans téléphone, identité masquée avant validation, ouverture de la popup vue par le coursier, un 2e coursier bloqué, un tiers refusé, refus + exclusion, expiration automatique, retrait, validation, positions dans les deux sens, codes de ramassage et de remise, annulation pendant une proposition) ; la 23e était un défaut du jeu d'essai (compte créé sans nom). 8 tests automatiques ajoutés dans `personal-courier.spec.ts` .
- **Non vérifié** : un livreur express avec cette boucle (même code que le coursier personnel) et le comportement sur téléphone.

5octies. Distance des livraisons : vitesse du téléphone (`delivery.module.ts`)

- `POST /deliveries/:id/location` accepte `speed` (m/s) et applique `odometerStep` avec la vitesse, comme pour les courses, **pour les deux modes** (coursier personnel et livreur express), seulement à partir du ramassage (IN_PROGRESS). La réponse renvoie `traveledMeters` (nombre décimal, la colonne `Delivery.traveledMeters` est un `Float` : aucune perte de précision).
- **Migration** `20261017170000_delivery_odometer_cred` : colonne `Delivery.odoCred` (`DOUBLE PRECISION NOT NULL DEFAULT 0`) = mètres déjà comptés par la vitesse depuis le point d'ancrage, pour ne pas compter deux fois. Appliquée sur une base vide, sans écart avec le schéma.
- Le prix final du coursier personnel (`confirmDelivery`) utilise ces kilomètres ; le prix du livreur express ne change pas (fixé à la commande).
- Tests ajoutés (`personal-courier.spec.ts` , 35 au total) : petit déplacement à 1,2 m/s compté pour les deux modes, vitesse nulle = 0, avant le ramassage = 0. **Non vérifié sur un trajet réel**.

6. Qualité et limites

- **31 suites / 358 tests** verts, `tsc` sans erreur.
- SMS (OTP) : toujours aucune passerelle ; le mode simulation (code 0000) sert aux tests.
- À faire : renouveler le mot de passe de la base (une URL a été partagée dans une conversation) ; fichier `google-services.json` pour les notifications push.

Commit `8d82013c` : mobile seulement (retour possible pendant l'inscription), aucun changement côté serveur.

7. Commit du jour

Commit	Contenu
`2a2a8d82`	Distance précise (vitesse du téléphone) pour les livraisons, migration `20261017170000`

`f717e40b`	Proposition du coursier validée par le client (migration `20261017150000`, routes, expiration, temps réel, suivi du client)
`0afb4017`	Commissions détaillées, distance par la vitesse du téléphone, message des 15 %, rayon 20 km, colis réservés aux coursiers validés, nom du prestataire, script de spécialité
`88e49834`	Base des lieux élargie (17 022 lieux, 77 communes, Kpota de Calavi) ; rapports et couverture
`8ab4ce66`	Un coursier occupé ou réservé ne se voit plus proposer d'autres missions
`ecb4be03`	Inscription professionnelle par numéro seul ; tricycle réservé aux gros colis ; base nationale des lieux ; plaque partiellement masquée